SIMATS ENGINEERING
ASSESSMENT TOOL 2

COURSE NAME: Database Management Systems
COURSE CODE: CSA05

COURSE OUTCOME COVERED

CO2: *Apply the relational model, relational algebra, SQL query structures, and views to solve database querying and data manipulation problems.* (BL3, BL6)

Activity Tool : Scenario-Based Task (High)
Assessment Tool Weightage: 35%

Dr

QUESTIONS		Total Marks: 50	
Q.No	**Question**	**Bloom's Level**	**Marks**
1	Given the scenario of a School Management System, identify relations and write SQL to list all students with their class teacher and grade.	BL3 – Apply	**5**
2	A hospital wants to track patient appointments. Design the relational schema and write SQL to find doctors with more than 10 appointments this month.	BL3 – Apply	**5**
3	In an e-commerce scenario, write SQL queries using sub-queries to find products that have never been ordered.	BL3 – Apply	**5**
4	For a university database, write relational algebra expressions to find students enrolled in both 'DBMS' and 'OS' courses.	BL3 – Apply	**5**
5	Given a payroll scenario, create a view showing employees whose salary is below department average and write a query to update their salary by 10%.	BL6 – Create	**5**
6	In a banking scenario, apply JOIN operations to retrieve customer account details, transaction history, and branch information.	BL3 – Apply	**5**
7	A retail store needs daily sales summary. Write SQL with GROUP BY and HAVING to report total sales per category exceeding Rs. 10,000.	BL3 – Apply	**5**

8	For a library system scenario, construct tuple relational calculus expressions to find members who borrowed books from all genres.	BL3 – Apply	**5**
9	Given an inventory management scenario, define CHECK and FOREIGN KEY constraints and demonstrate their enforcement through SQL.	BL3 – Apply	**5**
10	Design a relational schema for an HR system and create materialized views for monthly attendance and performance summaries.	BL6 – Create	**5**

Code of Conduct:

I A.Chaatriksai (192571003) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:
A.Chaatriksai

RUBRICS FOR CALCULATION:

Scenario based assessment					
Criteria	**Weightage (%)**	**Level 4 (Exemplary)**	**Level 3 (Proficient)**	**Level 2 (Developing)**	**Level 1 (Beginning)**
Understanding of Scenario	20%	Demonstrates clear and complete understanding of the scenario and context	Shows good understanding with minor gaps	Partial understanding; misses key aspects	Misinterprets or fails to understand the scenario
Identification of Key Issues	20%	Accurately identifies all relevant issues and factors involved	Identifies most key issues with minor omissions	Identifies limited issues; some irrelevant points	Unable to identify key issues
Application of Concepts	25%	Applies appropriate concepts effectively to address the scenario	Applies concepts with minor errors	Limited or partially correct application	Unable to apply relevant concepts
Decision Making	20%	Makes well-reasoned, logical decisions with strong	Makes reasonable decisions with some	Makes weak decisions with limited justification	No clear decisions made or rationale

		rationale	justification		provided
Clarity of Response	15%	Response is clear, structured, and logically presented	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand

1. School Management System

```
-- Create Tables
CREATE TABLE Teacher (
    TeacherID INT PRIMARY KEY,
    TeacherName VARCHAR(50)
```

```
);

CREATE TABLE Class (
   ClassID INT PRIMARY KEY,
   ClassName VARCHAR(20),
   TeacherID INT,
   FOREIGN KEY (TeacherID) REFERENCES Teacher(TeacherID)
);

CREATE TABLE Student (
   StudentID INT PRIMARY KEY,
   StudentName VARCHAR(50),
   Grade CHAR(2),
   ClassID INT,
   FOREIGN KEY (ClassID) REFERENCES Class(ClassID)
);

-- Insert Data
INSERT INTO Teacher VALUES
(1,'Ramesh'),
(2,'Priya');

INSERT INTO Class VALUES
(101,'10A',1),
(102,'10B',2);

INSERT INTO Student VALUES
(1,'Sahil','A',101),
(2,'Rahul','B',102);

-- Query
SELECT s.StudentName,c.ClassName,t.TeacherName,s.Grade
FROM Student s
JOIN Class c ON s.ClassID=c.ClassID
JOIN Teacher t ON c.TeacherID=t.TeacherID;
```

OUTPUT :

```
mysql>
mysql> -- Display all students with class teacher and grade
mysql> SELECT
    ->     s.StudentID,
    ->     s.StudentName,
    ->     c.ClassName,
    ->     t.TeacherName,
    ->     s.Grade
    -> FROM Student s
    -> JOIN Class c
    -> ON s.ClassID = c.ClassID
    -> JOIN Teacher t
    -> ON c.TeacherID = t.TeacherID;
+-----------+-------------+-----------+-------------+-------+
| StudentID | StudentName | ClassName | TeacherName | Grade |
+-----------+-------------+-----------+-------------+-------+
|         1 | Sahil       | 10A       | Ramesh      | A     |
|         3 | Aman        | 10A       | Ramesh      | A+    |
|         2 | Rahul       | 10B       | Priya       | B     |
+-----------+-------------+-----------+-------------+-------+
3 rows in set (0.04 sec)
```

2. Hospital Appointment System

```
CREATE TABLE Doctor(
DoctorID INT PRIMARY KEY,
DoctorName VARCHAR(50)
);

CREATE TABLE Appointment(
AppointmentID INT PRIMARY KEY,
DoctorID INT,
AppointmentDate DATE,
FOREIGN KEY(DoctorID) REFERENCES Doctor(DoctorID)
);

SELECT d.DoctorName,COUNT(*) AS TotalAppointments
FROM Doctor d
JOIN Appointment a
ON d.DoctorID=a.DoctorID
WHERE MONTH(a.AppointmentDate)=MONTH(CURDATE())
AND YEAR(a.AppointmentDate)=YEAR(CURDATE())
GROUP BY d.DoctorID,d.DoctorName
HAVING COUNT(*)>10;
```

OUTPUT :

```
mysql>
mysql> -- Query: Doctors with more than 10 appointments this month
mysql> SELECT
    ->     d.DoctorID,
    ->     d.DoctorName,
    ->     COUNT(a.AppointmentID) AS TotalAppointments
    -> FROM Doctor d
    -> JOIN Appointment a
    -> ON d.DoctorID = a.DoctorID
    -> WHERE MONTH(a.AppointmentDate) = MONTH(CURDATE())
    ->   AND YEAR(a.AppointmentDate) = YEAR(CURDATE())
    -> GROUP BY d.DoctorID, d.DoctorName
    -> HAVING COUNT(a.AppointmentID) > 10;
+-----------+------------+-------------------+
| DoctorID | DoctorName | TotalAppointments |
+-----------+------------+-------------------+
|        1 | Dr. Kumar  |                11 |
+-----------+------------+-------------------+
1 row in set (0.06 sec)
```

3. E-Commerce Products Never Ordered

```
CREATE TABLE Product(
ProductID INT PRIMARY KEY,
ProductName VARCHAR(50)
);

CREATE TABLE Orders(
OrderID INT PRIMARY KEY,
ProductID INT,
FOREIGN KEY(ProductID) REFERENCES Product(ProductID)
);

SELECT ProductName
FROM Product
WHERE ProductID NOT IN
(
SELECT ProductID
FROM Orders
);
```

OUTPUT :

```
mysql> -- Query: Products Never Ordered
mysql> SELECT ProductID, ProductName
    -> FROM Product
    -> WHERE ProductID NOT IN
    -> (
    ->     SELECT ProductID
    ->     FROM Orders
    -> );
+-----------+-------------+
| ProductID | ProductName |
+-----------+-------------+
|       103 | Keyboard    |
|       104 | Monitor     |
+-----------+-------------+
2 rows in set (0.01 sec)
```

4. Relational Algebra

```
-- Create Database
CREATE DATABASE IF NOT EXISTS UniversityDB;
USE UniversityDB;

-- Drop tables if they exist
DROP TABLE IF EXISTS Enrollment;
DROP TABLE IF EXISTS Course;
DROP TABLE IF EXISTS Student;

-- Create Student Table
CREATE TABLE Student (
   StudentID INT PRIMARY KEY,
   StudentName VARCHAR(50)
);

-- Create Course Table
CREATE TABLE Course (
   CourseID INT PRIMARY KEY,
   CourseName VARCHAR(50)
);

-- Create Enrollment Table
CREATE TABLE Enrollment (
   StudentID INT,
   CourseID INT,
   PRIMARY KEY (StudentID, CourseID),
   FOREIGN KEY (StudentID) REFERENCES Student(StudentID),
   FOREIGN KEY (CourseID) REFERENCES Course(CourseID)
);
```

```
-- Insert Sample Data
INSERT INTO Student VALUES
(1,'Sahil'),
(2,'Rahul'),
(3,'Aman');

INSERT INTO Course VALUES
(101,'DBMS'),
(102,'OS'),
(103,'Java');

INSERT INTO Enrollment VALUES
(1,101),
(1,102),
(2,101),
(3,102),
(3,103);

-- Students enrolled in BOTH DBMS and OS
SELECT s.StudentID, s.StudentName
FROM Student s
JOIN Enrollment e ON s.StudentID = e.StudentID
JOIN Course c ON e.CourseID = c.CourseID
WHERE c.CourseName IN ('DBMS','OS')
GROUP BY s.StudentID, s.StudentName
HAVING COUNT(DISTINCT c.CourseName) = 2;
```

OUTPUT :

```
mysql> SELECT s.StudentID,
    ->        s.StudentName
    -> FROM Student s
    -> JOIN Enrollment e
    -> ON s.StudentID = e.StudentID
    -> JOIN Course c
    -> ON e.CourseID = c.CourseID
    -> WHERE c.CourseName IN ('DBMS','OS')
    -> GROUP BY s.StudentID, s.StudentName
    -> HAVING COUNT(DISTINCT c.CourseName) = 2;
+-----------+-------------+
| StudentID | StudentName |
+-----------+-------------+
|         1 | Sahil       |
+-----------+-------------+
1 row in set (0.03 sec)
```

5. Payroll View

```
CREATE TABLE Employee(
```

```
EmpID INT PRIMARY KEY,
EmpName VARCHAR(50),
DepartmentID INT,
Salary DECIMAL(10,2)
);

CREATE VIEW LowSalaryEmployees AS
SELECT *
FROM Employee e
WHERE Salary<
(
SELECT AVG(Salary)
FROM Employee
WHERE DepartmentID=e.DepartmentID
);

UPDATE Employee
SET Salary=Salary*1.10
WHERE EmpID IN
(
SELECT EmpID
FROM LowSalaryEmployees
);
```

OUTPUT :

```
mysql> -- Display Updated Employee Table
mysql> SELECT * FROM Employee;
+-------+---------+--------------+----------+
| EmpID | EmpName | DepartmentID | Salary   |
+-------+---------+--------------+----------+
|     1 | Sahil   |          101 | 25000.00 |
|     2 | Rahul   |          101 | 35000.00 |
|     3 | Aman    |          102 | 20000.00 |
|     4 | Priya   |          102 | 30000.00 |
|     5 | Neha    |          103 | 40000.00 |
+-------+---------+--------------+----------+
5 rows in set (0.00 sec)
```

6. Banking JOIN

```
CREATE TABLE Customer(
CustomerID INT PRIMARY KEY,
CustomerName VARCHAR(50)
);

CREATE TABLE Branch(
BranchID INT PRIMARY KEY,
BranchName VARCHAR(50)
```

);

CREATE TABLE Account(
AccountID INT PRIMARY KEY,
CustomerID INT,
BranchID INT,
FOREIGN KEY(CustomerID) REFERENCES Customer(CustomerID),
FOREIGN KEY(BranchID) REFERENCES Branch(BranchID)
);

CREATE TABLE Transactions(
TransactionID INT PRIMARY KEY,
AccountID INT,
Amount DECIMAL(10,2),
FOREIGN KEY(AccountID) REFERENCES Account(AccountID)
);

SELECT c.CustomerName,
a.AccountID,
t.TransactionID,
t.Amount,
b.BranchName
FROM Customer c
JOIN Account a ON c.CustomerID=a.CustomerID
JOIN Transactions t ON a.AccountID=t.AccountID
JOIN Branch b ON a.BranchID=b.BranchID;

OUTPUT :

```
    ->     t.TransactionID,
    ->     t.TransactionDate,
    ->     t.TransactionType,
    ->     t.Amount,
    ->     a.Balance
    -> FROM Customer c
    -> JOIN Account a
    -> ON c.CustomerID = a.CustomerID
    -> JOIN Branch b
    -> ON a.BranchID = b.BranchID
    -> JOIN Transactions t
    -> ON a.AccountID = t.AccountID;
+------------+--------------+-----------+------------------+------------+---------------+-----------------+-------------
----+----------+----------+
| CustomerID | CustomerName | AccountID | BranchName       | BranchCity | TransactionID | TransactionDate | TransactionT
ype | Amount   | Balance  |
+------------+--------------+-----------+------------------+------------+---------------+-----------------+-------------
----+----------+----------+
|          1 | Sahil        |      1001 | Chennai Branch   | Chennai    |             1 | 2026-08-01      | Deposit
    | 10000.00 | 50000.00 |
|          1 | Sahil        |      1001 | Chennai Branch   | Chennai    |             2 | 2026-08-02      | Withdrawal
    |  5000.00 | 50000.00 |
|          2 | Rahul        |      1002 | Bangalore Branch | Bangalore  |             3 | 2026-08-03      | Deposit
    |  7000.00 | 30000.00 |
+------------+--------------+-----------+------------------+------------+---------------+-----------------+-------------
----+----------+----------+
3 rows in set (0.00 sec)
```

7. Retail Store GROUP BY

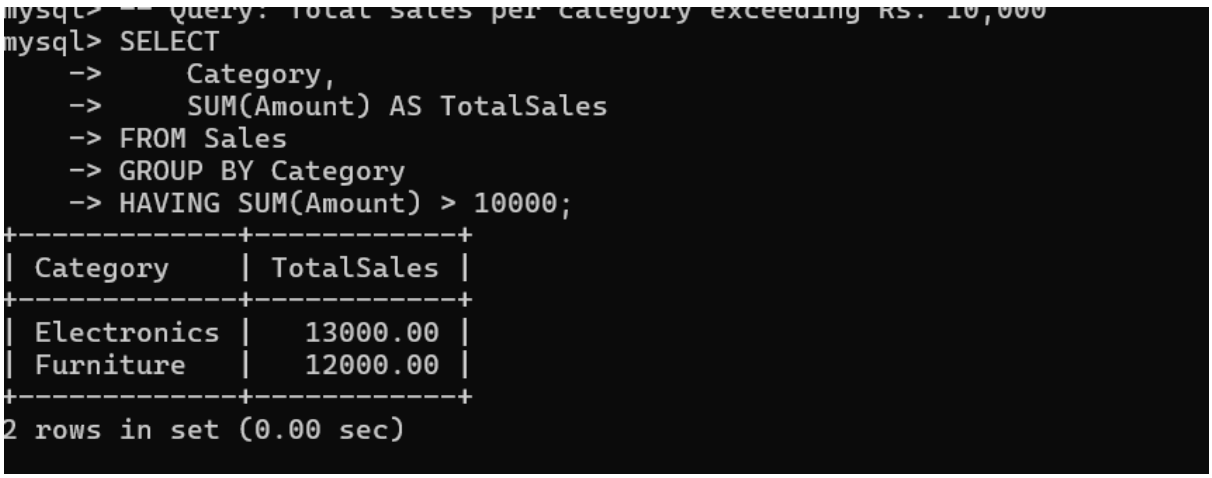
CREATE TABLE Sales(

```
SaleID INT PRIMARY KEY,
Category VARCHAR(50),
Amount DECIMAL(10,2)
);

INSERT INTO Sales VALUES
(1,'Electronics',6000),
(2,'Electronics',7000),
(3,'Clothing',4000),
(4,'Clothing',3000),
(5,'Furniture',12000);

SELECT Category,
SUM(Amount) AS TotalSales
FROM Sales
GROUP BY Category
HAVING SUM(Amount)>10000;
```

OUTPUT :



```
mysql> -- Query: Total sales per category exceeding Rs. 10,000
mysql> SELECT
    ->     Category,
    ->     SUM(Amount) AS TotalSales
    -> FROM Sales
    -> GROUP BY Category
    -> HAVING SUM(Amount) > 10000;
+-------------+------------+
| Category    | TotalSales |
+-------------+------------+
| Electronics |   13000.00 |
| Furniture   |   12000.00 |
+-------------+------------+
2 rows in set (0.00 sec)
```

8. Tuple Relational Calculus

```
-- Create Database
CREATE DATABASE IF NOT EXISTS LibraryDB;
USE LibraryDB;

-- Drop tables if they exist
DROP TABLE IF EXISTS Borrow;
DROP TABLE IF EXISTS Book;
DROP TABLE IF EXISTS Member;
DROP TABLE IF EXISTS Genre;

-- Create Tables
```

```
CREATE TABLE Member (
   MemberID INT PRIMARY KEY,
   MemberName VARCHAR(50)
);

CREATE TABLE Genre (
   GenreID INT PRIMARY KEY,
   GenreName VARCHAR(50)
);

CREATE TABLE Book (
   BookID INT PRIMARY KEY,
   BookName VARCHAR(50),
   GenreID INT,
   FOREIGN KEY (GenreID) REFERENCES Genre(GenreID)
);

CREATE TABLE Borrow (
   MemberID INT,
   BookID INT,
   PRIMARY KEY (MemberID, BookID),
   FOREIGN KEY (MemberID) REFERENCES Member(MemberID),
   FOREIGN KEY (BookID) REFERENCES Book(BookID)
);

-- Insert Sample Data
INSERT INTO Member VALUES
(1,'Sahil'),
(2,'Rahul');

INSERT INTO Genre VALUES
(1,'DBMS'),
(2,'Programming'),
(3,'Networks');

INSERT INTO Book VALUES
(101,'Database Systems',1),
(102,'C Programming',2),
(103,'Computer Networks',3);

INSERT INTO Borrow VALUES
(1,101),
(1,102),
(1,103),
```

(2,101);

-- Members who borrowed books from ALL genres
SELECT m.MemberID, m.MemberName
FROM Member m
JOIN Borrow b ON m.MemberID = b.MemberID
JOIN Book bk ON b.BookID = bk.BookID
GROUP BY m.MemberID, m.MemberName
HAVING COUNT(DISTINCT bk.GenreID) = (SELECT COUNT(*) FROM Genre);

OUTPUT :

```
mysql> SELECT
    ->     m.MemberID,
    ->     m.MemberName
    -> FROM Member m
    -> JOIN Borrow br
    -> ON m.MemberID = br.MemberID
    -> JOIN Book b
    -> ON br.BookID = b.BookID
    -> GROUP BY m.MemberID, m.MemberName
    -> HAVING COUNT(DISTINCT b.GenreID) = (
    ->     SELECT COUNT(*) FROM Genre
    -> );
+----------+------------+
| MemberID | MemberName |
+----------+------------+
|        1 | Sahil      |
+----------+------------+
1 row in set (0.05 sec)
```

9. Inventory Constraints
CREATE TABLE Supplier(
SupplierID INT PRIMARY KEY,
SupplierName VARCHAR(50)
);

CREATE TABLE Product(
ProductID INT PRIMARY KEY,
ProductName VARCHAR(50),
Price DECIMAL(10,2) CHECK(Price>0),
SupplierID INT,
FOREIGN KEY(SupplierID)
REFERENCES Supplier(SupplierID)
);

INSERT INTO Supplier VALUES
(1,'ABC Suppliers');

```
INSERT INTO Product VALUES
(101,'Keyboard',800,1);

-- Invalid (CHECK fails)
-- INSERT INTO Product VALUES(102,'Mouse',-500,1);

-- Invalid (FOREIGN KEY fails)
-- INSERT INTO Product VALUES(103,'Monitor',7000,5);
```

OUTPUT :

```
mysql>
mysql> -- Display Data
mysql> SELECT * FROM Supplier;
+------------+---------------+
| SupplierID | SupplierName  |
+------------+---------------+
|          1 | ABC Suppliers |
|          2 | XYZ Suppliers |
+------------+---------------+
2 rows in set (0.00 sec)

mysql> SELECT * FROM Product;
+-----------+-------------+--------+----------+------------+
| ProductID | ProductName | Price  | Quantity | SupplierID |
+-----------+-------------+--------+----------+------------+
|       101 | Keyboard    | 800.00 |       20 |          1 |
|       102 | Mouse       | 500.00 |       50 |          2 |
+-----------+-------------+--------+----------+------------+
2 rows in set (0.00 sec)
```

10. HR System (Materialized View Equivalent in MySQL)

```
CREATE TABLE Employee(
EmpID INT PRIMARY KEY,
EmpName VARCHAR(50)
);

CREATE TABLE Attendance(
AttendanceID INT PRIMARY KEY,
EmpID INT,
AttendanceDate DATE,
Status VARCHAR(10),
FOREIGN KEY(EmpID) REFERENCES Employee(EmpID)
);

CREATE TABLE Performance(
PerformanceID INT PRIMARY KEY,
EmpID INT,
Score INT,
FOREIGN KEY(EmpID) REFERENCES Employee(EmpID)
```

);

-- Monthly Attendance Summary
CREATE TABLE MonthlyAttendance AS
SELECT EmpID,
MONTH(AttendanceDate) AS Month,
COUNT(*) AS TotalDaysPresent
FROM Attendance
WHERE Status='Present'
GROUP BY EmpID,MONTH(AttendanceDate);

-- Monthly Performance Summary
CREATE TABLE MonthlyPerformance AS
SELECT EmpID,
AVG(Score) AS AverageScore
FROM Performance
GROUP BY EmpID;

SELECT * FROM MonthlyAttendance;
SELECT * FROM MonthlyPerformance;

OUTPUT :

```
mysql>
mysql> -- Display Summary Tables
mysql> SELECT * FROM MonthlyAttendance;
+-------+-------+-----------------+
| EmpID | Month | TotalAttendance |
+-------+-------+-----------------+
|     1 |     8 |               2 |
|     2 |     8 |               1 |
+-------+-------+-----------------+
2 rows in set (0.00 sec)

mysql> SELECT * FROM MonthlyPerformance;
+-------+--------------+
| EmpID | AverageScore |
+-------+--------------+
|     1 |      87.5000 |
|     2 |      75.0000 |
|     3 |      80.0000 |
+-------+--------------+
3 rows in set (0.00 sec)
```